

Bioinformática

**Revisões de programação em Python:
Parte 1**

Variáveis e objetos

Variáveis são entidades (nomes) que podem referenciar valores distintos durante a execução de um programa

Variáveis podem guardar valores “simples” de tipos pré-definidos: valores **numéricos** (inteiros ou decimais), valores **lógicos** (verdadeiro ou falso), **strings** (texto)

O python não define os tipos das variáveis explicitamente, sendo “assumidos” pelo contexto da computação

Variáveis podem também guardar dados mais complexos. Estes podem ser implementados por **objetos** de diversas **classes**, pré-definidas ou programadas pelo utilizador (e.g. strings ou lists)

Funções

Permitem realizar operações de processamento de dados

Embora sendo mais flexíveis, são semelhantes a funções matemáticas, recebem valores (variáveis) de entrada e retornam (opcionalmente) um dado resultado



Funções

- Existem funções pré-definidas, podendo também definir-se novas funções (como iremos ver mais tarde ...)
- **Invocação**: permite executar funções
 - invoca-se a função passando os argumentos de entrada na forma *função(argumentos)* e obtendo-se o resultado.
 - algumas funções podem ser aplicadas sobre um objeto, usando-se a notação *objeto.função(argumentos)*
 - neste último caso, o conteúdo do objeto sobre o qual se aplica a função pode ser alterado

Variáveis: atribuição

Através da **atribuição** associa-se o nome de uma variável a um dado conteúdo (pode ser um valor "simples" ou uma estrutura mais complexa)

Sintaxe do python: **nome_variavel = valor**

(note-se que para testar se uma variável é igual a um valor se usa o operador **==**).

Exemplo: **a = 5** significa que o valor da variável de nome *a* passa a ser de 5
(**a == 5** testa se a variável *a* tem valor 5)

A instrução **del** permite remover variáveis

Variáveis de tipos primitivos: numéricas

Guardam valores **inteiros** (**int**) ou **reais** (**float**) dentro de uma determinada gama

Aos seus valores podem aplicar-se **operadores aritméticos**: **+**, **-**, *****, **/**, exponenciação (******), divisão inteira(**//**), resto da divisão inteira (**%**)

Usando algum destes operadores seguido de **=**, atualizamos o valor da variável – por exemplo, as seguintes linhas são equivalentes:

```
a += 3
```

```
a = a+3
```

Existem também funções pré-definidas: Módulo (**abs**), Arredondar (**round**), ...

Package **math** inclui um numeroso lote de funções matemáticas e científicas úteis (**sin**, **tan**, **cos**, **sqrt**, **exp**, **factorial**, **log**, **tanh**, ...)

Variáveis numéricas: exemplos - shell

```
>>> x = 5
>>> y = 4
>>> x + y
9
>>> x * y
20
>>> x / y
1.25
>>> x // y
1
>>> z = 25
>>> z % x
0
>>> z % y
1
>>> x ** y
625
>>> type(x)
<class 'int'>
>>> del x
>>> x
...
NameError: name 'x' is not defined
```

```
>>> z = 3.0
>>> type(z)
<class 'float'>
>>> abs(-3)
3
>>> round(3.4)
3.0
>>> float(2)
2.0
>>> int(3.4)
3
>>> int(4.6)
4
>>> int(-2.3)
-2
>>> 0.0000000000000001
1e-14
>>> 2.3e-3
0.0023
```

```
>>> import math
>>> math.sqrt(4)
2.0
>>> math.sin(0.5)
0.479425538604203
>>> math.log(x)
1.6094379124341003
>>> math.pi
3.141592653589793
>>> math.tan(math.pi)
-1.2246467991473532e-16
>>> math.e
2.718281828459045
>>> math.exp(1)
2.718281828459045
>>> math.log(math.e)
1.0
```

Variáveis lógicas e operadores de comparação

Guardam apenas dois tipos de valores lógicos (Booleanos): **True** ou **False**

Aos seus valores podem aplicar-se **operadores lógicos**: **and**, **or**, **not**

Podem ser obtidas como resultado de um processo de comparação com os operadores **=** (igual a), **!=** (diferente de), **>** (maior do que), **>=** (maior ou igual a), **<** (menor do que), **<=** (menor ou igual a)

Podem ser resultado da aplicação de funções ou operadores a outros tipos de variáveis

Variáveis lógicas: exemplos - shell

```
>>> x = True
>>> y = False
>>> x and y
False
>>> x or y
True
>>> not y
True
>>> True and False
False
>>> not 0
True
>>> 0 or 1
1
```

```
>>> v = 1000
>>> v == 1000
True
>>> v != 1000
False
>>> v > 500
True
>>> v > 800 and v < 1200
True
>>> v < 800 or v > 1200
False
>>> v <= 1000
True
```

Variáveis do tipo **list**

Permitem armazenar e processar sequências (ordenadas) de valores

Definir uma variável x do tipo list: ***x = [1,2,3,4,5]***

Aceder a uma posição/ modificar o seu valor:

1ª posição: ***x[0] = 10*** (primeira posição tem índice 0)

elem = x[3] atribuir à var. elem o valor na 4ª posição do lista x

Listas podem ter valores de tipos diferentes: ***y = [1, "A", 2, "C"]***

Lista vazia: ***vazia = []***

Variáveis do tipo **list**: funções que recebem listas

Função **len** permite retornar o nº de elementos de uma lista

Funções **max** e **min** calculam o maior/ menor valor da lista

Função **sum** retorna a soma dos elementos da lista

Função **sorted** retorna a lista ordenada

Operadores **in** / **not in** permitem testar se elemento está na lista

```
>>> x = [1, 7, 4, 3, 5, 2]
>>> len(x)
6
>>> max(x)
7
>>> min(x)
1
>>> sum(x)
22
>>> sorted(x)
[1, 2, 3, 4, 5, 7]
>>> 5 in x
True
>>> 10 in x
False
```



Atenção: estas funções não mudam a própria lista e podem ser aplicadas a outras estruturas sequenciais (containers) que estudaremos em seguida !!

Variáveis do tipo **list**: funções aplicáveis sobre objetos

append: adicionar elementos a listas (no final) `x.append(3)`

insert: adicionar elemento numa dada posição `x.insert(2, "A")`

extend: adicionar elementos de uma lista a outra lista `x.extend([1,2,3])`

index: posição da primeira ocorrência de um elemento `x.index(3)`

count: conta nº ocorrências de um elemento na lista `x.count(3)`

pop: remove elemento (numa dada posição se for dada, ou no final em caso contrário) `x.pop(2)`

remove: remove um dado elemento nomeado `x.remove("A")`

sort: ordena lista `x.sort()`

reverse: inverte lista `x.reverse()`



Atenção: a maior parte destas funções mudam a própria lista !!

Variáveis do tipo list: exemplos

```
>>> x = [1, 7, 4, 3, 5, 2]
>>> x.append(6)
>>> x
[1, 7, 4, 3, 5, 2, 6]
>>> x.index(5)
4
>>> x.extend([9,8])
>>> x.insert(1,10)
>>> x
[1, 10, 7, 4, 3, 5, 2, 6, 9, 8]
>>> x.pop()
8
>>> x.reverse()
>>> x
[9, 6, 2, 5, 3, 4, 7, 10, 1]
>>> x.sort()
>>> x
1, 2, 3, 4, 5, 6, 7, 9, 10]
```

```
>>> l2 = [1,'a',2,'b']
>>> l2.sort()
>>> l2
[1, 2, 'a', 'b']
>>> l2.insert(2,'d')
>>> l2
['c', 3, 'd', 'a', 1]
>>> l2.append(1)
>>> l2
['c', 3, 'd', 'a', 1, 1]
>>> l2.count(1)
2
>>> l2.pop()
'b'
>>> l2
[1, 2, 'a']
>>> l2.remove(2)
>>> l2
[1, 'a']
>>> [0]*5
[0, 0, 0, 0, 0]
```

Variáveis do tipo **tuple**

Funcionam de forma similar às *lists* mas são imutáveis (não podem ser alterados)

Indexação, concatenação semelhantes às listas; também são containers e por isso as funções/operadores **len**, **max**, **min**, **sorted**, **sum**, **in** funcionam como nas listas

Funções que tentam mudar tuplos dão erro

```
>>> p = (2, 5, 4)
>>> p[1]
5
>>> p[0:2]
(2, 5)
>>> len(p)
3
>>> 5 in p
True
>>> p.append(4)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'tuple' object has no ...
```

Variáveis do tipo **set**

Funcionam de forma similar às *lists* mas representam conjuntos e por isso não permitem repetidos

Operadores **&** e **|** usados para interseção e união; operador **<=** testa se conjunto está contido noutro

Indexação, concatenação semelhantes às listas; também são containers e por isso as funções/operadores **len**, **max**, **min**, **sorted**, **sum**, **in** funcionam como nas listas

Funções próprias aplicadas sobre conjuntos: **add**, **remove**, **pop**

```
>>> set([1,2,3])
{1, 2, 3}
>>> set([1,2,3]) & set([1,2,4])
{1, 2}
>>> set([1,2,3]) | set([1,2,4])
{1, 2, 3, 4}
>>> set([2,3]) <= set([1,2,3,4])
True
>>> s = set([1,3,2,4])
>>> max(s)
4
>>> sum(s)
10
>>> 5 in s
False
>>> s.add(5)
>>> s
{1, 2, 3, 4, 5}
>>> s.pop()
1
>>> s
{2, 3, 4, 5}
>>> s.remove(4)
>>> s
{2, 3, 5}
```

Variáveis textuais: tipo **string**

Variáveis do tipo **string** – guardam sequências de caracteres (letras, números, etc)

São definidas como sequências de caracteres entre ‘ ‘ ou “ “

Partilham algumas funções e sintaxe com as listas, pois ambos são estruturas sequenciais (**containers**)

As strings são imutáveis, i.e. o seu valor não se altera depois de serem criadas

Variáveis tipo **string**: funções e operadores

Junção de duas strings s1 e s2: **s1+s2**

Comprimento de uma string: **len(s)**

Caracter na posição i de uma string s: **s[i]**

Substituir todas as ocorrências de C1 por C2 na string s

s.replace("C1","C2")

Converter para minúsculas ou maiúsculas **s.upper()**

s.lower()

Contar o nº de vezes que um padrão p ocorre numa sequência s

s.count(p)

Variáveis tipo **string**: funções e operadores

Operador **in** permite verificar se um padrão fixo existe numa string

Função **find** permite retornar a posição da primeira ocorrência de um padrão numa string (retorna -1 se não for encontrada)

Função **count** conta nº de ocorrências de um padrão numa string

Função **replace** permite substituir todas as ocorrências de um padrão por um novo padrão

Indexação em listas e strings: slicing

Algumas operações de acesso a elementos de lists e strings fazem-se da mesma forma

Intervalos de valores:

`s[i:j]` – todos os elementos entre as posições `i` e `j-1`

`s[i:]` – todos os elementos entre as posições `i` e o final

`s[:i]` – todos os elementos entre o início e a posição `i-1`

Passo: pode acrescentar-se um 3º argumento que representa “saltos” na sequência

`s[i:j:k]` – todos os elementos entre as posições `i` e `j-1` saltando de `k` em `k` posições

Se passo negativo a sequência é percorrida pela ordem inversa (exemplo:

`s[::-1]` inverte a sequência `s`)

Representação de sequências

Computacionalmente, assumiremos que as sequências biológicas são representadas como **strings** (sequências de caracteres)

DNA e RNA: alfabeto com 4 letras: A, C, G, T para DNA e A, C, G, U para RNA

Proteínas representadas por um alfabeto de 20 letras, cada uma representando um aminoácido

Alfabetos IUPAC

Symbol	Name	Nucleotides represented
A	Adenine	A
C	Cytosine	C
G	Guanine	G
T	Thymine	T
U	Uracil	U

Symbol	Name
A	Alanine
C	Cysteine
D	Aspartic acid
E	Glutamic acid
F	Phenylalanine
G	Glycine
H	Histidine
I	Isoleucine
L	Lysine
M	Methionine
N	Asparagine
P	Proline
Q	Glutamine
R	Arginine
S	Serine
T	Threonine
V	Valine
T	Tryptophan
Y	Tyrosine

Variáveis do tipo string: exemplos

```
>>> seq = 'AATAGATCGA'
>>> len(seq)
10
>>> seq[5]
'A'
>>> seq[4:7]
'GAT'
>>> seq.count('A')
5
>>> seq2 = "ATAGATCTAT"
>>> seq+seq2
'AATAGATCGAATAGATCTAT'
>>> '1' + "1"
'11'
>>> seq.replace('T','U')
'AAUAGAUCGA'
```

```
>>> seq[::2]
'ATGTG'
>>> seq[::-2]
'ACAAA'
>>> seq[5:1:-2]
'AA'
>>> seq.lower()
'aatagatcga'
>>> seq.lower()[2:]
'tagatcga'
>>> seq.lower()[2:].count('c')
1
>>> c=seq.count("C")
>>> g=seq.count("G")
>>> float(c+g)/len(seq)*100
30.0
```

```
>>> "TAT" in "ATGATATATGA"
True
>>> "TATC" in "ATGATATATGA"
False
>>> str = "ATGATATATGA"
>>> "TAT" in str
True
>>> "TATC" in str
False
>>> str.find("TAT")
4
>>> str.find("TATC")
-1
>>> str.count("TA")
2
```

Variáveis do tipo dicionário

Os dicionários guardam pares (chave, valor), sendo que não existem chaves repetidas

Implementa conceito de função finita (ou tabela de Hash)

Valores de chaves e valores podem ter tipos distintos (numéricos, string, etc)

Criação de um dicionário:

```
dict = {'A': 'Ala', 'C': 'Cys', 'S': 'Ser'}
```

Acesso/modificação de elementos:

```
dict['C']
```

```
dict['G'] = "Glu"
```

Variáveis do tipo dicionário: funções

dict.keys() – devolve lista com as chaves

dict.values() – devolve lista com os valores

dict.get(k, v) – devolve valor associado à chave k; idêntico a **dict[k]** mas permite definir valor por omissão v se chave não existir

del dict[k] – remove elemento do dicionário

dict.clear() – remove todos os elementos do dicionário

k in dict – verifica existência de uma chave

Variáveis tipo dicionário: exemplos

```
>>> dic = {"Cao":"Mamifero", "Polvo":"Molusco", "Cobra":"Reptil"}
>>> dic['Cao']
'Mamifero'
>>> dic['Camarao']= 'Molusco'
>>> dic
{'Cobra': 'Reptil', 'Camarao': 'Molusco', 'Cao': 'Mamifero',
 'Polvo': 'Molusco'}
>>> len(dic)
4
>>> dic.keys()
dict_keys(['Cobra', 'Camarao', 'Cao', 'Polvo'])
>>> list(dic.keys())
['Cobra', 'Camarao', 'Cao', 'Polvo']
>>> "Homem" in dic
False
>>> "Cao" in dic
True
>>> del dic["Cobra"]
>>> dic
{'Camarao': 'Molusco', 'Cao': 'Mamifero', 'Polvo': 'Molusco'}
>>> list(dic.values())
['Molusco', 'Mamifero', 'Molusco']
```

Estrutura típica de um programa

Entrada de dados

- do standard input (teclado)
- leitura de ficheiros, etc

Processamento de dados

conjunto de instruções que permite trabalhar os dados lidos e gerar os resultados desejados

Apresentação dos resultados (saída)

- standard output (monitor)
- escrita de ficheiros



Programas com esta estrutura podem ser criados usando o IDE (e.g. Spyder) e corridos, apresentando os resultados na linha de comandos

Leitura de dados do *standard input*

A função básica de entrada de dados é a função `input`

Resultado desta função é atribuído a uma variável que guarda o valor lido na forma de uma string:

```
x= input()
```

Função pode receber como parâmetro uma mensagem (string) para ser imprimida no ecrã informando o utilizador que se espera uma entrada de dados

```
x= input("Introduza sequencia:")
```

Valor recebido é uma string, podendo ser convertido para um tipo adequado

```
i = int(input("Introduza valor inteiro:"))  
f = float(input("Introduza numero real:"))
```

Escrita de dados no *standard output*

- A forma mais básica de informar um utilizador de alguma resultado de um programa passa pela sua impressão no ecran
- Função **print** permite escrever dados no ecran, podendo incluir-se textos constantes e/ou valores de variáveis/ objetos

```
print("Hello world")
```

```
print("O valor de x:", x)
```

```
print("Coordenadas do ponto", x, " , ", y)
```

Programa: junção de duas sequências

```
seq1 = input("Sequencia 1:").upper()  
seq2 = input("Sequencia 2:").upper()
```

```
seq = seq1 + seq2
```

```
print("Sequencia final:" + seq)
```

Leitura das sequências (strings)

Concatenação (operador +)

Saída do resultado final

Programa: calcular a hipotenusa

```
import math

a = float(input("Comprimento do cateto A: "))
b = float(input("Comprimento do cateto B: "))

c = math.sqrt(a ** 2 + b ** 2)

print("Comprimento da hipotenusa:", c)
```

Importação do package math necessária para usar a função **sqrt**

Função **float** converte string lida para variável numérica

Operador ****** exponenciação

Instruções condicionais

if *condição:*

instrução 1
instrução 2
(...)

else:

instrução 1
instrução 2
(...)

Condição: valor lógico
(variável, expressão, etc)

Bloco de instruções
executado se a condição for
verdadeira

Bloco de instruções
executado se a condição for
falsa



Note a **indentação** (obrigatória): blocos de instruções dentro dos ramos **if** / **else** são colocados um tab à direita

Condições

Condição tem que ter um valor lógico – **True** ou **False**

Possíveis condições podem incluir:

- variáveis lógicas e constantes;
- operadores de comparação: `==`, `<`, `>`, `>=`, `<=`
- operadores lógicos: conjunção (**and**), disjunção (**or**), negação (**not**)
- funções com resultado lógico
- expressões com valores lógicos
- valores inteiros

```
>>> x = 5
>>> x < 6
True
>>> x > 10
False
>>> not x > 2
False
>>> x == 3
False
>>> x != 5
False
```

Programa: calcular o maior de dois valores inteiros

```
x = int(input("X = "))
y = int(input("Y = "))

if x > y:
    m = x
else:
    m = y

print ("Maior: ", m)
```

Lê os valores e converte as strings para inteiros com a função **int**

Variável m guarda o menor dos dois valores

Instruções condicionais

```
if condição1:  
    instrução 1  
    (...)
```

```
elif condição2:  
    instrução 1  
    (...)
```

```
(elif ...)
```

```
else:  
    instrução 1  
    (...)
```

Bloco de instruções executado se a condição 1 for **verdadeira**

Bloco de instruções executado se a condição 1 for **falsa** e a condição 2 **verdadeira**

Bloco de instruções executado se as condições forem todas **falsas**

Programa: minicalculadora

```
x = float(input("X = "))
y = float(input("Y = "))
op = input("Operador:")

if op=="+":
    res = x + y
elif op=="-":
    res = x - y
elif op=="*":
    res = x * y
elif op=="/":
    res = x / y
else: print("Erro")

print(res)
```

Lê dois valores reais e um operador (string)

Exercício:

Imprimir também a indicação da operação efetuada